

VaultProof

Prove solvency. Reveal nothing.

DoraHacks — Flare Summer Signal
Bounty 2: Confidential Compute Apps
Deployed on Flare Coston2 (chain ID 114)

Contents

1 Project overview	2
2 Short product description	2
3 Target user	2
4 How it helps day-to-day life	2
4.1 Why it generalises	3
5 Full flow — six stages	3
5.1 The four details that carry the security	3
6 How the project uses Flare	3
7 What was newly built	4
7.1 Base used as-is	4
7.2 Built during the program	4
7.3 Three real bugs found and fixed	5
8 Deployment details	5
8.1 Live evidence	5
9 Technical execution	5
10 Honest limitations	6
11 Roadmap	6
12 Distribution and testing	6
13 Repository, team and licence	6

1. Project overview

Project name	VaultProof
Bounty	Bounty 2 — Confidential Compute Apps (Flare Confidential Compute)
One line	Prove solvency. Reveal nothing.
Network	Flare Coston2 testnet, chain ID 114
Repository	https://github.com/Lakshay-Deol/VaultProof
Team	Prashant Thakur
Licence	MIT

2. Short product description

VaultProof is a confidential solvency oracle on Flare Confidential Compute. A borrower seals a read-only exchange API key to a public key that only a hardware enclave holds. The enclave decrypts it, queries the exchange, prices the holdings with Flare's own FTSO feeds, and reduces everything to a coarse solvency tier plus a nullifier and an expiry.

What reaches the chain is a wallet address, a tier, an expiry, a nullifier and a code-hash measurement — never the API key, the exchange name, the asset mix, the exact balance, or the account identifier. A `LendingPool` contract reads that tier to extend undercollateralised credit.

FDC brings public data on-chain. FCC brings private conclusions on-chain.

FDC's security comes from replication, and replication is incompatible with secrecy — you cannot ask ten verifiers to check a balance behind one API key without giving ten verifiers the API key. That gap is what this fills.

3. Target user

Crypto-holding borrowers locked out of undercollateralised credit. Someone with 2,000 USDC on Flare and \$52,000 on an exchange, who cannot borrow 8,000 USDC because the chain cannot see off-chain wealth and there is no safe way to prove it.

Lending protocols that want a solvency signal without custody of user data. A pool that would extend a larger cap given evidence of off-chain assets, but does not want to hold, process, or be liable for exchange credentials.

4. How it helps day-to-day life

Maya holds 2,000 USDC on-chain and \$52,000 on Kraken. Her boiler dies; she needs 8,000 USDC for six weeks. She is good for it many times over — but every on-chain lender sees 2,000 USDC and lends her about 1,600.

Her options today	What it costs her
Sell the exchange position	Taxable event, loses a position she wanted to keep, to solve a six-week gap
Exchange loan desk	KYC, custody, jurisdiction limits, days to weeks
Give an API key to a scoring service	Their ops team reads her live balances; their breach becomes her breach

Each trades away the position, the time, or the privacy. VaultProof trades away none: she seals a read-only key, the enclave publishes T3, the pool raises her cap.

4.1 Why it generalises

The same machine answers other questions. Swap the final reducer and you get:

- **Proof of income for renting** — an income band, instead of three months of bank statements listing every merchant and habit.
- **Proof of reserves for custodians** — solvency without publishing the treasury layout to competitors and attackers.
- **Private KYC** — “over 18, not sanctioned, resident of X”, computed over documents no database ever holds.

The general shape is *a private fact you can prove but not safely share*.

5. Full flow — six stages

Each stage is gated on the last in code, not in the UI.

#	Stage	Public	Sealed
1	Connect	wallet address	—
2	Verify	measurement, enclave pubkey	operator cannot substitute a key
3	Seal	ciphertext length	API key, secret, exchange
4	Anchor	keccak256(blob), tx hash	blob never enters calldata
5	Enclave	“processing”	balance, asset mix, account id
6	Attested	tier, expiry, nullifier, build	everything else, permanently

5.1 The four details that carry the security

1. **The signature is verified in the user’s browser**, against Google’s published keys — not on our backend. A signature our own server checks for you proves nothing to you. `alg: none` is rejected outright.
2. **The enclave’s public key is inside the signed token**, passed as the attestation `eat_nonce`. A relay that swaps in its own key fails on the user’s machine.
3. **Hash first, ciphertext second**. Putting ciphertext in calldata publishes it permanently, so a future key compromise would retroactively expose every credential ever submitted.
4. **Pricing happens inside the enclave, from FTSO**. If it trusted the exchange’s own USD figure, anyone running a server that speaks the Kraken wire format could mint themselves a top tier.

After all six stages the chain knows roughly **2.5 bits** about Maya’s wealth. That is the entire public footprint of a \$52,000 credit decision.

6. How the project uses Flare

The test of a meaningful integration is what breaks without it, so that is the column that matters.

Component	Role	What breaks without it
FCC / Confidential Space	Runs the handler in a VM the host cannot read; the launcher — outside the container's control — signs a statement of which image booted.	There is no version of this on a normal server. Somebody's ops team reads live credentials, and the entire product is that nobody can.
FTSO	Prices holdings inside the enclave, from the chain the loan settles on.	A self-hosted "exchange" could inflate itself into the top tier.
Measurement registry	On-chain whitelist of trusted code hashes, checked before encrypting.	"Trust the code, not the operator" becomes unverifiable.
Coston2	Hosts registry, pool and sender; anchors requests, enforces caps at drawdown.	The enclave's output is a dashboard receipt, not a credit decision.

7. What was newly built

7.1 Base used as-is

Flare's `fce-extension-scaffold`, `flare-foundry-starter` and `flare-viem-starter`.

7.2 Built during the program

- **Five Solidity contracts**, deployed and Blockscout-verified. `SolvencyRegistry` enforces expiry in the read path, handles nullifier collisions, and re-checks the measurement at write time. `LendingPool` applies tier caps that extend rather than replace collateral. `VaultProofInstructionSender` handles the anchor, the TEE write path and FCC routing. Plus the measurement registry and a test USDC token.
- **The solvency enclave** in Go — HPKE unseal, Kraken HMAC-SHA512 signed reads, FTSO pricing resolved through the Flare contract registry, HMAC nullifier, zeroize in defers. Credentials are held in `[]byte`, never `string`, because Go strings cannot be scrubbed.
- **Real hardware attestation** — the enclave requests a token from the Confidential Space launcher over its unix socket, binds its X25519 key as the `eat_nonce`, and reports the signed `image_digest` as its measurement. A fresh token per request, not one minted at boot and replayed.
- **Real quote verification** — the browser fetches Google's Confidential Computing key set and verifies the token signature with WebCrypto, enforcing issuer and expiry.
- **The whole frontend** — 4 pages, 6 stages, mock/live adapter seams, a state machine, real in-browser HPKE, and the design system. The directory was empty at the start of the program.
- **Both live adapters** — `viem/wagmi` against the deployed contracts, and the enclave client against `/quote` and `/sealed`.
- **The attack simulator** — stage 2 ships a "Try breaking it" control that runs a substituted relay key and an unwhitelisted build against the same verifier the honest path uses. Both hard-stop, with no override.

- **73 tests** — 25 Go, 11 Foundry, 33 web unit, 4 live against Coston2.

7.3 Three real bugs found and fixed

1. The scaffold's `teeutils.ToHash` right-pads a string into a `bytes32`, while the contract and frontend use `keccak256`. The two disagree completely and the failure is silent — the exact vanishing-instruction bug the spec warns about.
2. Blob delivery was keyed by SHA-256 while the browser anchors `keccak256`, so the enclave would never have found the blob.
3. Attestation check 2 verified the token's *binding* but never its *signature*, so a forged token with matching fields would have passed all three checks.

8. Deployment details

Flare Coston2 testnet, chain ID 114. All five contracts verified on Blockscout.

Contract	Address
SolvencyRegistry	0xD653bE4c296E2462D22254953D2Aaa7D4DA1917C
LendingPool	0x3b7c700cd2d812348de61BD13b28e601C661b5Da
VaultProofInstructionSender	0x45540745B838F6f3feC76E662b5539BcB82339c3
TeeMeasurementRegistry	0xe1788fF42Fc5a5B4012d5af6f8B51fe3a3eF36f7
MockUSDC (tUSDC)	0x459c634EE948f6D486b714E06C1F186034F2e7A4

8.1 Live evidence

- T3 attestation delivered — `0xd99378e7a668ff52eef4e032902cd8e2d4df99401b908fb316ac686af73ceb4f`
- Borrowed 15,000 tUSDC — `0x66fde48b7de4c0871b23662fa966e2a13d984d773f90fcb0c0d316e54bd02b46`
- A 50,000 attempt reverted with `over cap` — the contract refuses rather than trusting the front end.

9. Technical execution

Each part can be checked independently:

```
cd web && npm run dev
    localhost:3000, mock mode, no wallet or funds needed
cd extension/go && go run ./cmd
    the enclave on :8080
curl localhost:8080/quote
    measurement, enclave pubkey, attestation mode
go run ./cmd/smoke
    seals as the browser does, attests, checks replay
cd web && npm run test:live
    4 tests against the real Coston2 deployment
```

`cmd/smoke` proves the two HPKE implementations agree: it fails at `unseal`: if they ever drift, and at `query`: if the credential simply is not a real Kraken key — so the message tells you which.

10. Honest limitations

- **Hardware mode has never executed.** The attestation code is real and on the real path — on Confidential Space it fetches a genuine AMD SEV-SNP-backed token and the browser verifies it against Google. But it has not been run on that hardware, so no claim is made that it has. Locally it reports simulated mode and the UI renders it as such. Every failure branch fails closed to simulated rather than overclaiming.
- **Attestation freshness.** A quote proves the code that booted, not the code running at request time. Per-request tokens narrow this; nothing closes it.
- **Side channels.** AMD SEV has a published history of them. Adequate at moderate size; not a claim of perfection.
- **Exchange trust.** If Kraken lies about a balance, VaultProof faithfully attests a lie. The system proves what a named source said, not what is true.
- **Governance.** Whoever controls whitelisting is a trusted role. The system is not trustless end to end and does not claim to be.

11. Roadmap

1. **Run it on Confidential Space** — build reproducibly, list the resulting image digest on the measurement registry, deploy, and serve hardware mode.
2. **Reproducible-build verifier CLI** — so strangers can rebuild the image and check the hash themselves.
3. **Multi-TEE quorum** — use the cosigner threshold already in the FCC instruction params; require k -of- n enclaves to agree.
4. **Revocation** — an explicit revoke path and a kill switch for a superseded build.
5. **More sources** — Coinbase, Binance, then bank APIs behind one credential schema.
6. **New reducers** — proof of income, proof of reserves, private KYC assertions.
7. **Governance hardening** — whitelist control to a multisig, then to a lender-governed set.
8. **Audit, Songbird, mainnet.**

12. Distribution and testing

No user acquisition yet — the project is pre-public-demo. Testing to date is the 73-test suite plus on-chain runs against the live deployment. The honest statement is that this has been validated technically, not yet with real users.

13. Repository, team and licence

Repository	https://github.com/Lakshay-Deol/VaultProof
Design spec	docs/design-spec.pdf — threat model §10, build plan §11, demo script §12
Team	Prashant Thakur
Licence	MIT

Still to fill in: the public demo URL and the recorded video walkthrough.